

What is the Spring framework?

- The Spring Framework is a mature, powerful and highly flexible framework focused on building web applications in Java.
- One of the core benefits of Spring is that it takes care of most of the low-level aspects of building the application to allow us to actually focus on features and business logic.
- Another strong point is that, while the framework is quite mature and well-established, it's very actively maintained and has a thriving dev community. This makes it quite up to date and aligned with the Java ecosystem right now.

Why should Java programmers use the Spring framework?

- **Usability:** Spring is very easy to use. It provides various configuration options for developers to start and then configure exactly what they need. Projects like Spring Boot have made bootstrapping a complex Spring project almost trivial. Not to mention, it has excellent documentation and tutorials to help anyone get on-boarded.
- **Modularity:** Spring boot is highly modular in nature. Developers can optionally include one or more Spring projects depending upon the need. Moreover, they can use other frameworks like Hibernate or Struts as well!
- **Testability:** Spring at the core advocates and supports Test Driven Development (TDD). Spring application is mostly composed of POJOs which naturally makes unit testing relatively much simpler.
- **Maturity:** Over the years, it's become mature enough to become a default solution for most common problems faced in the development of large scale enterprise applications. What's even more exciting is how actively it's being developed and maintained. Support for new language features and enterprise integration solutions are being developed every day.
- **Community Support:** Spring is an open source led by Pivotal Software and backed by a large consortium of organizations and individual developers. Hundreds of thousands of enterprise apps are built using spring.

What is Inversion of Control?

- Inversion of Control is a principle in software engineering which transfers the control of objects or portions of a program(or object creation and management) to a container or framework.
- Contrary to traditional programming, in which our custom code makes calls to a library, IoC enables a framework to take control of the flow of a program and make calls to our custom code. To enable this, frameworks use abstractions with additional behavior built in. e.g. @Controller automatically routes traffic to controllers defined in the controller class.
- We can achieve Inversion of Control through various mechanisms such as: Strategy design pattern, Service Locator pattern, Factory pattern, and **Dependency Injection (DI)**.

Benefits of IoC?

- Decoupling the execution of a task from its implementation.
- Easily switch between different implementations.
- Greater modularity.
- Ease of testing by isolating a component or mocking its dependencies.

What is the Dependency Injection ?

- Dependency injection is a pattern we can use to implement IoC, where the control being inverted is setting an object's dependencies.
- Connecting objects with other objects, or "injecting" objects into other objects, is done by an assembler rather than by the objects themselves.

How does spring supports IoC ?

- Spring supports IoC through ApplicationContext.
- An IoC container is a common characteristic of frameworks that implement IoC.
- In the Spring framework, the interface ApplicationContext represents the IoC container. The Spring container is responsible for instantiating, configuring and assembling objects known as beans, as well as managing their life cycles.
- Several implementations of the ApplicationContext interface are provided by Spring e.g. WebApplicationContext, ClassPathXmlApplicationContext etc.
- In order to assemble beans, the container uses configuration metadata, which can be in the form of XML configuration or annotations.
- If required, we can manually instantiate application context e.g.

```
ApplicationContext context  
= new ClassPathXmlApplicationContext("applicationContext.xml");
```

- When you create an instance of AnnotationConfigApplicationContext and provide it with one or more configuration classes, it scans these classes for the @Bean annotations and other relevant annotations. It then initializes and manages the beans defined in these classes, setting up their dependencies and managing their lifecycle.

How do you configure Spring Framework?

- Nowadays, Spring Framework is most configured using annotations.

Can we use more than one configuration file for our Spring project?

- Yes. However, all of them should be imported into the main application config which is used by the program.
- spring.config.additional-location or spring.config.location fields can be used in config file to point to location of other additional configs.

What types of dependency injection are supported by Spring Framework?

- Dependency Injection in Spring can be done through constructors, setters or fields.
- **Constructore based:** In the case of constructor-based dependency injection, the container will invoke a constructor with arguments each representing a dependency we want to set.
- **Setter based:** For setter-based DI, the container will call setter methods of our class after invoking a no-argument constructor or no-argument static factory method to instantiate the bean.

- **Field based:** In case of Field-Based DI, we can inject the dependencies by marking them with an @Autowired annotation.

Pros and cons of field based DI?

Pros: Simple Cons:

- This method uses reflection to inject the dependencies, which is costlier than constructor-based or setter-based injection.
- It's really easy to keep adding multiple dependencies using this approach. If we were using constructor injection, having multiple arguments would make us think that the class does more than one thing, which can violate the Single Responsibility Principle.

When do you use Setter and Constructor Injection, the pros and cons?

- Constructor based DI should be used when an object's dependencies are not optional, and they must be initialized with their dependencies.
- Setter injection when dependencies are optional for the object to work for short time.

Setter vs Constructor DI?

Setter	Constructor
Setter injection in Spring uses setter methods like setDependency() to inject dependency on any bean managed by Spring's IOC container.	Cconstructor injection uses the object's constructor to inject dependency on any Spring-managed bean.
Since the setter method has a name like setDependency(...), it is more readable in spring's applicationcontext.xml file	Since conctructor di uses an index to inject the dependency, it's less readable wrt setter injection and you need to refer to either Java documentation or code to find which index corresponds to which property.
Setter DI doesn't ensures that DI is injected when obj. is instantiated.	Constructor DI makes sure that dependency is injected before object is instantiated.
Security: Dependencies can be overrided, reinstered because setter is there.	Dependencies in obj. cannot be overrided.

Factory vs DI Pattern?

Factory	Dependency Injection
When using a factory your code is still actually responsible for creating objects.	By DI you outsource that responsibility to another class or a framework, which is separate from your code
Your code must request for the object needed(pull)	Required objects are injected to code(push)

Factory	Dependency Injection
Tight coupling: In the case of the Factory design pattern, the client class is responsible for getting instance of objects from the factory class. This makes client and factory class tightly coupled.	In Dependency Injection, the client class has no clue about how his dependencies are created and managed. It only knows about dependencies. Hence it's loosely coupled.
Unit testing is difficult because of tight coupling, we have to test client class, factory etc. together.	Easy unit testing. You just need to mock the dependency and inject it into an object you want to test, no clutter or boilerplate is required.
Less flexible.	More flexible. Can use different di frameworks like spring google guice etc.

What are the different modules in spring?

Spring has seven core modules

1. The Core container module
2. Application context module
3. AOP module (Aspect Oriented Programming)
4. JDBC abstraction and DAO module
5. O/R mapping integration module (Object/Relational)
6. Web module
7. MVC framework module

What is the difference between Spring MVC and Spring core?

- The Spring MVC is part of the Spring framework, which helps you to develop Java web applications using model web controller pattern.
- On the other hand, Spring Core provides the Dependency injection and Inversion of Control. The Spring Container is part of Spring core.
- Both functionalities come in different libraries.
- If you are developing just a core Java application using Spring, then you just need Spring Core.
- But if you are developing a Web application, then you need spring-mvc library.
-

What is AOP? How is it implemented in Spring?

- AOP is a programming paradigm that aims to increase modularity by allowing the separation of cross-cutting concerns.
- It does this by adding additional behavior to existing code without modifying the code itself.
- Spring AOP, aspects are implemented using regular classes or regular classes annotated with the @Aspect annotation. In other words by using JDK Proxies or CGLib proxies.

What's aspect?

- An aspect is a modularization of a concern that cuts across multiple classes. Unified logging can be an example of such cross-cutting concern.

What's joinpoint?

- A Joinpoint is a point during the execution of a program, such as the execution of a method or the handling of an exception.

What's advice?

- An Advice is an action taken by an aspect at a particular Joinpoint. Different types of advice include "around," "before," and "after."
- In Spring, an Advice is modelled as an interceptor, maintaining a chain of interceptors around the Joinpoint.

What's pointcut ?

- A Pointcut is a predicate that helps match an Advice to be applied by an Aspect at a particular JoinPoint.
- We often associate the Advice with a Pointcut expression, and it runs at any Joinpoint matched by the Pointcut.

What's component scanning in spring ?

- Component scanning is a Spring framework mechanism that automatically detects and registers beans (components, services, repositories, controllers) in the application context by searching the classpath for annotated classes. It eliminates the need for manual XML or Java configuration for every bean, using annotations like `@Component`, `@Service`, and `@ComponentScan`.

@ComponentScan without arguments: `@ComponentScan` without arguments tells Spring to scan the current package and all of its sub-packages.

@SpringBootApplication annotation is a combination of three annotations:

1. `@Configuration`
2. `@EnableAutoConfiguration`
3. `@ComponentScan`

@ComponentScan with arguments: Spring provides a convenient way to specify one/multiple package names in which it should look for components. To do so, we need to use a string array.

```
@ComponentScan(basePackages =  
{"com.baeldung.componentscan.springapp.animals",  
"com.baeldung.componentscan.springapp.flowers"})
```

Alternatively, since spring 4.1.1, we can use a comma, a semicolon, or a space to separate the packages list:

```
@ComponentScan(basePackages =
    "com.baeldung.componentscan.springapp.animals;com.baeldung.componentscan.sp
    ringapp.flowers")
```

- We can use * in component scan too like we do in java import statements.
- We can exclude packages/classes to be scanned by using exclusion filters e.g.

```
@ComponentScan(excludeFilters =  
    @ComponentScan.Filter(type=FilterType.REGEX,  
        pattern="com\\.baeldung\\.componentscan\\.springapp\\.flowers\\.*"))
```

Why we should avoid putting the @Configuration class in the default package?

- We should avoid putting the `@Configuration` class in the default package (i.e. by not specifying the package at all). If we do, Spring scans all the classes in all jars in a classpath, which causes errors and the application probably doesn't start.

What is an ApplicationContext in Spring?

- The `ApplicationContext` is the central interface of the Spring Framework that represents the IoC container. It is responsible for instantiating, configuring, and managing the lifecycle of beans.
- It extends `BeanFactory` and adds advanced enterprise features such as:
 - Internationalization (message resolution)
 - Application event publishing
 - Environment abstraction (profiles and properties)
 - Resource loading
- It's defined in `org.springframework.context.ApplicationContext`

BeanFactory vs ApplicationContext?

Feature	BeanFactory	ApplicationContext
Basic Idea	Minimal IoC container	Advanced, feature-rich IoC container
Package	org.springframework.beans.factory	org.springframework.context
Bean Initialization	Lazy (default)	Eager (default for singleton beans)
Performance	Faster startup (less overhead)	Slightly slower (more features loaded)
Internationalization (i18n)	Not supported	Supported (MessageSource)
Event Handling	Not supported	Supported (ApplicationEventPublisher)
Environment & Profiles	Not supported	Supported

Feature	BeanFactory	ApplicationContext
Resource Loading	Basic	Advanced (classpath, file, URL, etc.)
Bean Post Processors	Must be registered manually	Automatically registered
Use Case	Lightweight apps, memory-constrained environments	Enterprise applications (most common choice)
Common Implementations	XmlBeanFactory (deprecated)	ClassPathXmlApplicationContext, AnnotationConfigApplicationContext

How are you going to create a new instance of an ApplicationContext?

- There are multiple ways to create an instance of ApplicationContext depending upon whether you are working in a core java application or a web application. For example, in core java applications, you can use ClassPathXmlApplicationContext to create instances of ApplicationContext. It looks for applicationContext.xml anywhere in the classpath (including JAR files).
- You can also use AnnotationConfigApplicationContext, if you are using Java annotations to configure spring application. It is also the newest and most flexible implementation.

e.g.

```
import org.springframework.context.ApplicationContext;
import
org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class GreetingConfiguration {

    @Bean
    public GreetingProvider provider() {
        return new GreetingMessageProvider();
    }

    @Bean
    public GreetingRenderer renderer() {
        GreetingRenderer renderer = new StandardOutMessageRenderer();
        renderer.setGreetingProvider(provider());
        return renderer;
    }
}

public class HelloWorldSpringAnnotated {

    public static void main(String[] args) {
        ApplicationContext ctx =
            new
            AnnotationConfigApplicationContext(GreetingConfiguration.class);
```

```
        GreetingRenderer mr = ctx.getBean("renderer",
GreetingRenderer.class);
        mr.render();
    }
}
```

Can you describe the lifecycle of a Spring Bean in an ApplicationContext?

1. Spring reads the bean configuration and metadata in the form of a BeanDefinition object is created for each bean.
2. All instances of BeanFactoryPostProcessor are invoked in sequence and are allowed an opportunity to alter the bean metadata.
3. After that, following steps are repeated for each bean in the container:
 1. An instance of the bean is created using the bean metadata.
 2. Properties and dependencies of the bean are set.
 3. Any instances of BeanPostProcessor are given a chance to process the new bean instance before and after initialization.
 4. Any methods in the bean implementation class annotated with `@PostConstruct`, or any `afterPropertiesSet` methods in bean, or any custom bean initialization method are invoked. This processing is performed by a BeanPostProcessor.
4. Bean is now created.
5. Now, coming to bean destruction cycle:
 1. Any methods in the bean implementation class annotated with `@PreDestroy` are invoked.
 2. Any destroy method in a bean implementation class implementing the `DisposableBean` interface is invoked. If the same destruction method has already been invoked, it will not be invoked again.
 3. Any custom bean destruction method is invoked.
 4. Bean is then destroyed.